

Reviewer Pack — Minimal Number of Algebraic Curves with Bounded Intersection...

Entry c3471d87796d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 23:13:23 UTC

Minimal Number of Algebraic Curves with Bounded Intersection and SAT Clause Subset Entropy

Entry ID: c3471d87796d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 23:13:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Intersection Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Subsets)

Statement:

For a given Boolean formula φ with m clauses, the minimal number of algebraic curves in a variety that intersects each clause exactly once is $\Theta(m \log n)$, where n is the number of variables in φ .

Rationale (proposer's reasoning):

Intersection theory in algebraic geometry provides a way to encode combinatorial problems like those in SAT. The conjecture suggests that the complexity of encoding clauses as intersections on curves can be related to the entropy of subsets of clauses, providing a novel perspective on SAT clause structure and complexity.

Taxonomy category: intersection_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6d8f1497e84392de

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean formulas with m clauses, the minimal number of algebraic curves intersecting each clause exactly once is within a factor of 2 from $\Theta(m \log n)$ and the subset entropy of all possible subsets of clauses is within a factor of 2 from $\Theta(m \log n)$. The conjecture is falsified if either condition fails for any seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "intersection theory" AND "algebraic curve" AND "SAT clause subset entropy" - "Boolean satisfiability" AND "algebraic geometry" AND "minimal curves" - "algebraic variety" AND "curves with bounded intersection" AND "SAT clause entropy"

Top relevant hits considered: - [s2:e1b53d0f6a0eba426d2786c6abd6795c4c136ed5] 2. Birational Geometry of Log Surfaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(m, n):
        clauses = []
        for _ in range(m):
            clause = set(random.sample(range(n), 2))
            clauses.append(clause)
        return clauses

    def calculate_subset_entropy(clauses):
        num_clauses = len(clauses)
        total_subsets = 1 << num_clauses
        entropy = 0
        for i in range(total_subsets):
            subset = [clauses[j] for j in range(num_clauses) if (i & (1 << j))]
            if subset:
                prob = Fraction(1, total_subsets)
                entropy -= prob * math.log2(prob)
        return entropy

    def calculate_minimal_curves(clauses):
        n = len(clauses[0])
        return n * math.log(n)

    m = random.randint(5, 30)
    n = random.randint(5, 30)
```

```

clauses = generate_formula(m, n)
subset_entropy = calculate_subset_entropy(clauses)
minimal_curves = calculate_minimal_curves(clauses)

return {
    "metric_name": "minimal_curves",
    "metric_value": minimal_curves,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(minimal_curves - m * math.log(n)) <= m * math.log(n) / 2 and abs(s
    "counterexample": "" if conjecture_holds else f"m={m}, n={n}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\", first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70a4293a.py", line 63, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_70a4293a.py", line 55, in run_trial

```

    "counterexample": "" if conjecture_holds else f"m={m}, n={n}"
    ~~~~~

```

NameError: name 'conjecture_holds' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that neither the support nor falsification conditions could be evaluated. | next: Re-run the test without errors to verify if the conjecture holds or not.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13772
2	preregistration	ollama_remote	glm4:latest	0	0	9679
3	novelty	ollama_remote	glm4:latest	0	0	8503
4	novelty	ollama_remote	glm4:latest	0	0	11898
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	53888
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9475
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9167
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7784
9	judge	ollama_remote	glm4:latest	0	0	20433

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 144599 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c3471d87796d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c3471d87796d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c3471d87796d.tar.gz (if generated)